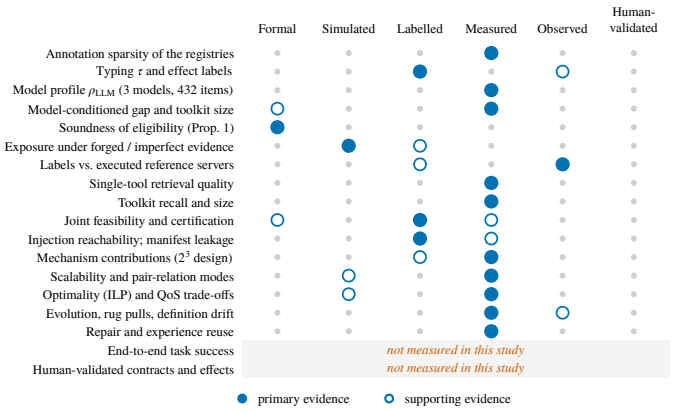


Figure 8. Evidence map: the class of evidence behind each claim evaluated in this section (filled: primary, open: supporting). Shaded rows are not measured.

Table 7. Data, splits and their use.

Data	Size	Use
Merged registry	569 / 76	tools / servers; 44 annotated; all experiments
LiveMCPBench tasks	95	multi-step test tasks (94 with gold tool)
MCPToolBench++ dev	60	tuning of every method (10 per category)
MCPToolBench++ test	300	single-tool retrieval (50 per category, 66 targets)
Security task set	155	95 test tasks + 60 dev prompts, two policies
Calibration suite	432	6 operations × 4 sizes × 2 languages × noise × context; 264 calibration / 168 test items, 3 models
Reference servers	6 / 38	servers / registry tools executed under tracing
Cloned registries	10^3–10^5	near-copies of real contracts (scaling)
Heterogeneous registries	10^3–10^5	generator varying servers, effects, sparsity, vocabulary, duplicates
Cover instances	80	8 families × 10; exact optima by ILP
QoS instances	120	4 roles × 4 providers; 4 weightings × 3 floors

8 Experimental Evaluation

This section evaluates PaMCP along the four research questions of Section 1. Section 8.1 fixes the data, the compared methods, the varied parameters and the statistical protocol. The next five subsections answer RQ1 (representation and model-conditioned gap, Section 8.2), RQ2 (trust under missing, forged and imperfect evidence, Section **??**), RQ3 (retrieval, toolkit quality and certification, Section **??**), the security behaviour and the contribution of each mechanism (Section **??**), and cost, scalability, adaptation and stability (RQ3/RQ4, Section **??**). Section **??** states the limits of the evidence. Every subsection gives the protocol, a results table, figures, the observations and their interpretation, and compares PaMCP with baselines wherever a baseline exists.

8.1 Experimental Setup

Evidence classes. The claims of the paper differ in what can establish them. We label every result with one of six evidence classes: *formal* (a proof in Sections 5–7), *simulated* (generated under stated assumptions, e.g. a verifier with a controlled error rate), *labelled* (scored against machine-generated reference labels), *measured* (obtained from the running implementation against benchmark gold or solver optima), *observed* (from controlled execution traces of real MCP servers) and *human-validated*. Fig. 8 maps each claim to the classes that support it. Two claims have no supporting evidence in this study: end-to-end task success of executed toolkits and human-validated contracts; no conclusion below rests on them.

Registries and tasks. Table 7 lists the data. The registry merges LiveMCPTool [1] and the tool set of MCP-ToolBench++ [2] by `server::name` (76 servers, 569 tools; Fig. 9a). Only 44 tools carry MCP annotations, all of them `readOnlyHint=true`, and one of these is not read-only (Fig. 9c); 119 of the 525 unannotated tools are not read-only. Multi-step tasks are the 95 public LiveMCPBench tasks (94 with a resolvable gold tool). Single-tool retrieval uses MCP-ToolBench++ requests: a 60-request development split (ten per category) for every tuning decision, and a disjoint test sample of 300 requests (50 per category, 66 target tools). Security and ablation experiments use 155 tasks (the 95 LiveMCPBench tasks and the 60 development prompts), each under its least-privilege policy and under a strict read-only policy; confirmatory tests use the test tasks only.

Reference labels. Neither benchmark labels effects or types. Labels J_A (Claude Sonnet, from name, description and schema under a fixed guide) give the typing τ and drive the simulated verifier bound to $h(t)$; labels J_B (Claude Opus, blind, separate guide) are used for scoring (Fig. 9b). They agree on state modification for 558 of 569 tools ($\kappa = 0.944$). Because every tool that is unsafe under J_B (120) is also unsafe under J_A (131), a verifier simulated from J_A can never admit a tool that J_B calls unsafe; we therefore also report the swapped direction (Section **??**) and compare both labellings with executed servers. All LLM outputs (analyst goal contracts, MCP-Zero requests, COLT training queries, ToolScope merge judgements, calibration answers) were recorded once and are replayed.

Compared methods. Table 8 lists the methods and what each may use. PaMCP combines three mechanisms that the ablations switch independently: the model-conditioned gap (G, Algorithm 1), provenance-sound eligibility (P, Section 5) and set-level reasoning with certification (S: backward closure, minimum-cost cover, certificate; Section 6). PaMCP-L is the relevance-first variant selected on the development split (lexical candidate ranking; structural matching only as a recorded, uncertain constraint). The baselines are adapted to the common registry and share one dense encoder (TF-IDF with truncated SVD); they are adaptations, not the published systems. Every retrieval method was tuned on the development split with a comparable grid (Table 9).

Varied parameters. Table 9 lists every parameter that an experiment varies, with its range and default; parameters not listed keep the values of Section 6.

Statistical protocol. Request-level metrics are resampled with target tools as clusters and task-level metrics with tasks as clusters; paired comparisons use two-sided sign-flip permutation tests on cluster sums, proportions use Wilson intervals, and all intervals are 95%. Two confirmatory families were fixed before the data they test were seen: seven comparisons F1–F7 (before the set-level experiments) and an amendment H1–H5 (committed, dated, before any test-set run of the revised analy-

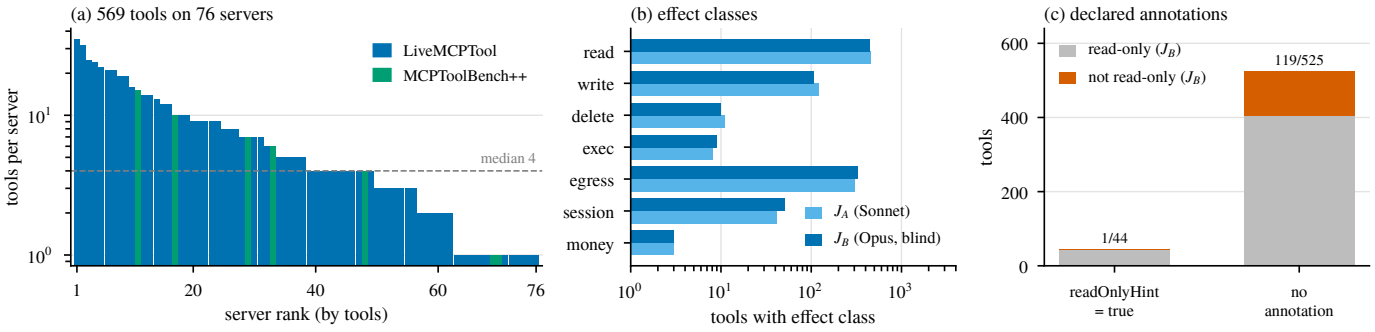


Figure 9. The evaluation registry. (a) Tools per server (log scale). (b) Effect classes under the two LLM labellings. (c) Declared annotations against the read-only status under J_B (numbers: not read-only / total).

Table 8. Compared methods: information used and tuning on the development split.

Method	Information used	Grid
PaMCP	analyst goal contracts, typing τ, evidence, profile, Eq. (6)	14
PaMCP-L	as PaMCP; lexical ranking first, types as constraint	(sel.)
RAG-MCP [3]	raw request, dense top-k	1
BM25 / RRF fusion	raw request, lexical / lexical+dense	12 / 4
MCP-Zero [4]	model-written requests, server routing	4
COLT [5]	query–scene–tool graph of 450 training queries	16
ToolScope [6]	LLM-confirmed tool merging, analyst subqueries	3
CMTF [7]	typed-state search, one tool per step	4
Per-subtask	analyst subgoals, one tool each	–
Full catalogue	all 569 tools	–

Table 9. Parameters varied in the experiments (default in bold).

Parameter	Values	Section
Candidate budget K	1, 5, **10**, 20, 40	**??**, **??**
Relevance weight w_{rel}	0, 1, 3, **10**	**??**
Profile threshold τ	0.6, 0.75, **0.9**; risk targets 0.05, 0.10	8.2
Input size n	10, 30, 40, 160, 300, 640, 3k, 30k	8.2
Forging servers δ	0, 0.05, 0.1, 0.2, 0.3 (5 seeds each)	**??**
Verifier false accept φ	0, 0.01, 0.02, **0.05**, 0.1, 0.2	**??**
Probe coverage κ	0.25, 0.5, 0.75, **1**	**??**
Trust threshold $\ell_{\min}$	0.2–0.95 (step 0.05), **0.65**	**??**
Poisoned descriptions	0, 10%, 30% of tools	**??**
Mechanisms G, P, S	on/off (2^3) + 5 components	**??**
Registry size	569, 10^3, 10^4, 10^5 tools	**??**
Pair relation	**eager**, partitioned, lazy	**??**
Cost weights $\beta, \gamma, \delta, \eta, \zeta$	0, 0.25, **0.5**, 1	**??**
Utility floor $u_{\min}$	0, 0.4, **0.5**, 0.6, 0.8, 0.9	**??**
Node budget (exact search)	**20,000**	**??**

ses), each with Holm correction and, for recall, a non-inferiority margin of 0.05. All other comparisons are exploratory and reported as such. Code, recorded LLM outputs, per-item results, run manifests (commit, input and label hashes, package versions) and a notebook that re-runs every experiment on a CPU runtime are available.[1]

8.2 RQ1: Representation and Model-Conditioned Gap

RQ1 asks whether the capability model can represent what tools and models do well enough to decide what a request needs. We

[1] `https://github.com/msellamiTN/taas-provenance-discovery`, branch `research/pamcp-final-evaluation`.

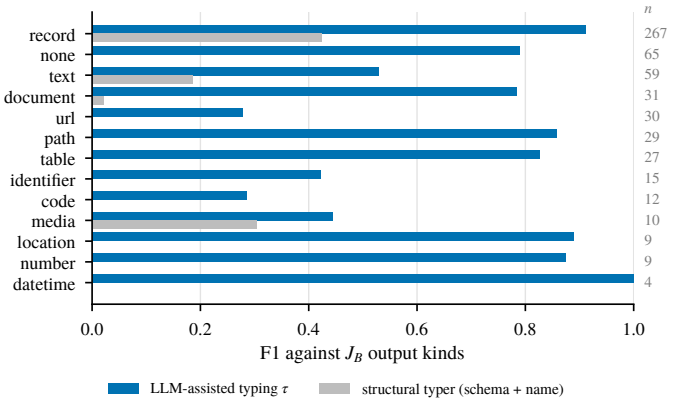


Figure 10. Typing agreement with J_B per output kind (kinds with at least four tools; right: support). The LLM-assisted typing τ is compared with a structural typer that uses schemas and names only.

test three of its ingredients: the typing τ on which structural reasoning relies, the measured reliability profile ρ_{LLM} that decides which operations the model keeps, and the gap of Algorithm 1 that turns the two into a toolkit.

Protocol. Typing is scored against the blind labels J_B (labelled evidence). The profile is measured on a calibration suite of 432 generated data operations (extraction, classification, counting, approximate sums, exact computation and joins; 10 to 640 records; English and French; clean or noisy text; isolated or embedded in a multi-step context). Claude Haiku, Sonnet and Opus answered every item by reading only, without tools, and stated beforehand, per cell, their probability of answering correctly. The threshold τ is chosen on 264 calibration items as the smallest value whose selective risk, the false-native rate among operations kept native, is at most the risk target, and is evaluated on 168 held-out items (measured evidence). One Opus batch of 45 items was interrupted by a usage limit; its items are excluded for Opus, not counted as errors. The gap is exercised on the review scenario of Fig. 1 for growing inputs.

Typing. The LLM-assisted typing agrees with J_B on 78.9% of output kinds ($\kappa = 0.718$, macro-F1 0.704) but on only 35.7% of complete types (kind and domain). A structural typer working from schemas and names alone reaches 22.8% ($\kappa = 0.070$, macro-F1 0.062) and recognises only records, text, documents and media at all (F1 at most 0.42; Fig. 10); τ itself is weakest

Table 10. Model-profile calibration: accuracy (Wilson 95% CI), self-assessment calibration, and the held-out behaviour of the threshold τ chosen for a 5% risk target (measured profile) versus the 64-item profile at $\tau = 0.9$. Cov.: share of operations kept native; FNR: false-native rate (errors among them).

				Self-assess.		Measured, risk 5%			64-item	
Model	n	Accuracy (%)	n=640	ECE	Brier	τ	Cov.	FNR	Cov.	FNR
Haiku	432	86.1 [82.5, 89.1]	67	0.39	0.30	0.21	83	3.6	21	0.0
Sonnet	432	91.4 [88.4, 93.7]	62	0.20	0.11	0.42	90	3.3	26	4.5
Opus	387	100.0 [99.0, 100.0]	100	0.38	0.20	0.90	100	0.0	–	–

on URLs, code and identifiers (F1 0.28–0.42). Types are therefore reliable at the kind level and unreliable at the domain level, which is where structural matching discriminates; this limits structural retrieval (Section **??**).

Capability boundaries. Table 10 and Fig. 11 show that the boundary between native and tool-requiring operations depends on the model and on the operation, not only on the input size. Haiku answers 86.1% of the items (95% CI 82.5–89.1%) and fails counting already at medium sizes (21% at $n = 40$, 6% at $n = 160$); Sonnet answers 91.4% (88.4–93.7%) and loses joins as they grow (71% at $n = 40$, 44% at $n = 160$); both fail approximate sums over 640 numbers. Opus answers all 387 items it was given, so its boundary lies beyond the suite. Language matters little (Haiku 88.0% in English against 84.3% in French; Sonnet 91.2% against 91.7%), whereas noise lowers Sonnet from 95.4% to 87.5% and exactness lowers Haiku from 92.5% (tolerant) to 78.1% (exact). A single profile per model family would thus misplace the boundary in both directions.

Measured against self-assessed reliability. All three models underestimate themselves (Fig. 12a): their stated probabilities average 0.47, 0.72 and 0.62 against accuracies of 0.86, 0.91 and 1.00 (ECE 0.39, 0.20 and 0.38). With the measured profile and a 5% risk target, the held-out false-native rate stays within the target for every model (3.6%, 3.3% and 0.0%) while 83%, 90% and 100% of the operations remain native (Fig. 12b). The 64-item profile used before this revision is safe but wasteful at $\tau = 0.9$: it keeps only 21% (Haiku) and 26% (Sonnet) of the operations native and sends most of the work the models perform correctly to tools. Self-assessment cannot replace measurement: at the same target it keeps 24% of Haiku's operations native. On the test and development tasks the measured profile overrules 12 of the analyst's 128 claims that an operation is native (9.4%).

Model-conditioned gap. On the review scenario the gap behaves as designed (Fig. 12c): with tolerant aggregation two tools suffice at $n = 30$ and 300, where the model keeps 60% of the goals, and six are needed at $n = 3{,}000$; exact aggregation adds two tools at the small sizes; at $n = 30{,}000$, and at $n = 3{,}000$ with $\tau = 0.6$, the engine abstains instead of returning an unsupported toolkit. Without gap analysis it abstains in every setting, because no registry tool serves the scenario's interpretive goals.

Interpretation. The representation supports model-conditioned discovery: what a request needs depends on the deployed model, and a measured profile with a risk-targeted threshold bounds the false-native rate on held-out operations at the chosen level. Two limits remain. The suite covers six data operations, not the whole transformation vocabulary of Definition 6, and Opus reaches its ceiling; profile stability across model updates was not measured. Typing is the weaker ingredient: its domain-level disagreement propagates to every structural decision evaluated next.

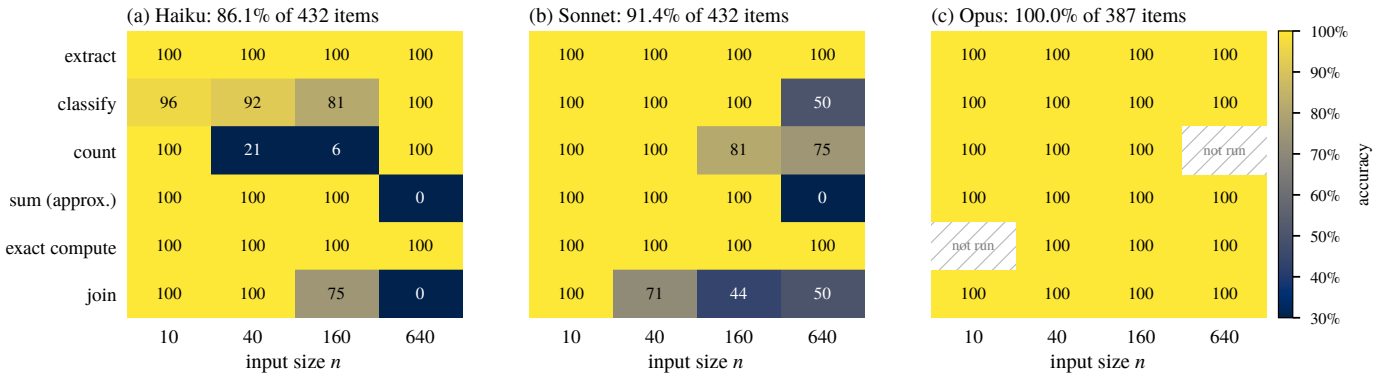


Figure 11. Measured reliability per operation and input size for three models (calibration suite, both splits; cells average languages, noise and context). Hatched cells belong to the interrupted Opus batch.

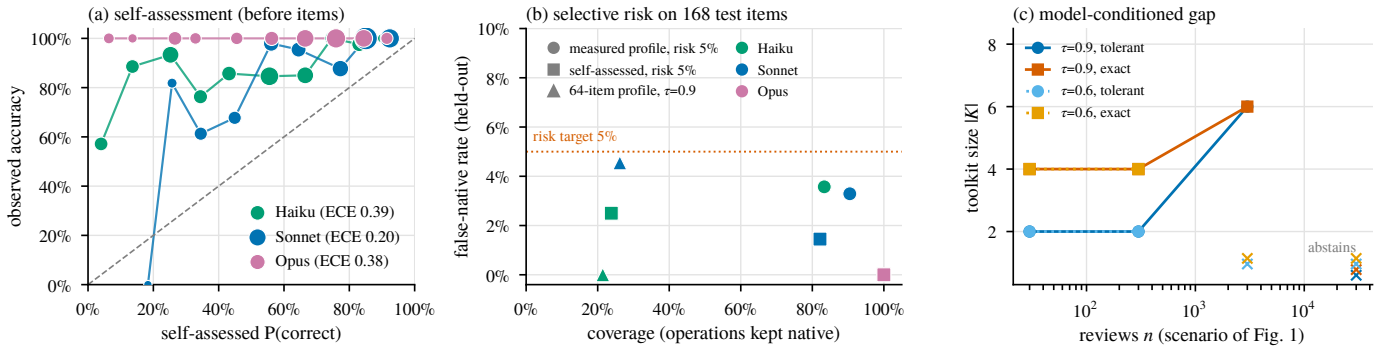


Figure 12. (a) Reliability diagram of the models' self-assessment, given before any item was seen (marker area: items per bin). (b) Held-out coverage and false-native rate of three profiles for each model. (c) Toolkit size of the review scenario against the input size, Sonnet profile; crosses mark abstentions.

REFERENCES

[1] G. Mo, W. Zhong, J. Chen, Q. Yuan, X. Chen, Y. Lu, H. Lin, B. He, X. Han, and L. Sun. LiveMCPBench: Can agents navigate an ocean of MCP tools? *arXiv preprint arXiv:2508.01780*, 2025.

[2] S. Fan, X. Ding, L. Zhang, and L. Mo. MCPToolBench++: A large scale AI agent model context protocol MCP tool use benchmark. *arXiv preprint arXiv:2508.07575*, 2025.

[3] T. Gan and Q. Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*, 2025.

[4] X. Fei, X. Zheng, and H. Feng. MCP-Zero: Active tool discovery for autonomous LLM agents. *arXiv preprint arXiv:2506.01056*, 2025.

[5] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-R. Wen. Towards completeness-oriented tool retrieval for large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*, pages 1930–1940. ACM, 2024.

[6] M. M. Liu, D. Garcia, F. Parllaku, V. Upadhyay, S. F. A. Shah, and D. Roth. ToolScope: Enhancing LLM agent tool use through tool merging and context-aware filtering. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 34095–34119, 2026.

[7] R. Suresh Babu and L. Ganesh Iyer. ToolChoiceConfusion: Causal minimal tool filtering for reliable LLM agents. arXiv:2606.06284, 2026.